Deep Reinforcement Learning

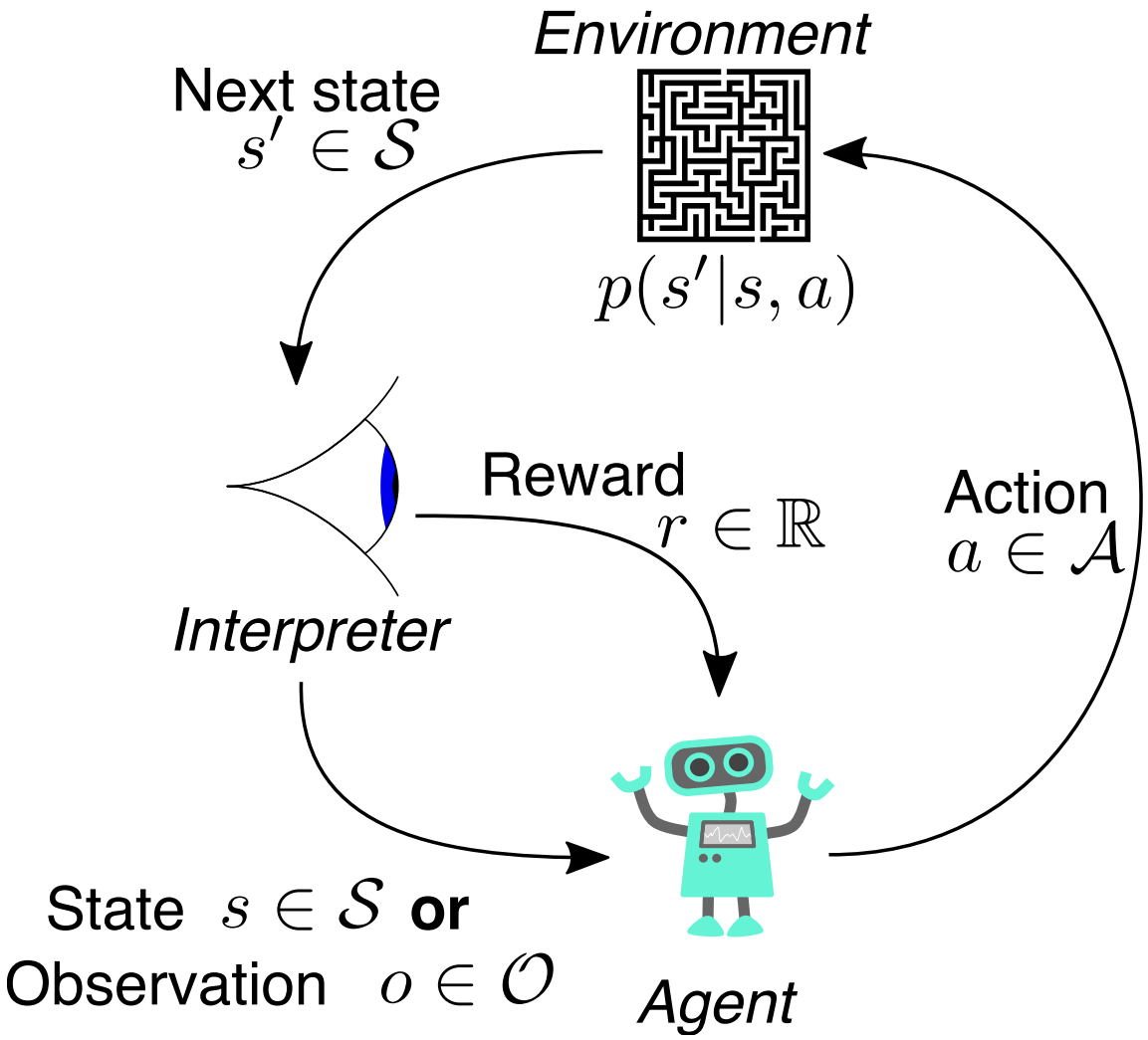
Markov Decision Processes

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

What is reinforcement learning?



In words

- An **agent** perceives the state s of its environment.
- It takes an **action** a according to a **policy** $\pi(a|s)$.
- The environment changes due to the action: $s \to s'$.
- The agent receives a **reward** r.

The goal

Some remarks on stochasticity

Stochasticity in reinforcement learning

All the pieces of our RL framework are generally subject to some ***randomness***

- the **policy** $\pi(a|s)$ is random (or, more precisely, a conditional probability)
  - given a state s, π denotes the probabilities of all the actions a we might take
  - if $\mathcal{A}$ is finite, then π is a *distribution* over $\{a_1, a_2, \dots, a_{|\mathcal{A}|}\}$
  - if $\mathcal{A}$ is infinite (e.g., $a \in [0, 1]$), then π is the probability density function over $\mathcal{A}$
- the **state transition** $p(s'|s, a)$ is also random
- the **state observation** o might be subject to noise
- the **reward** inherits this randomness, even if we were to define it deterministically

$\Rightarrow$ we need to deal with ***random variables***

Random variables

A **random variable** X is a *measurable function* $X : \Omega \to E$ from a *sample space* Ω as a set of possible *outcomes* to a measurable space E (i.e., we can assign a number to each outcome).

The **realization** x is the value of $X(\omega)$ after a single experiment $\omega \in \Omega$.

Some examples

- we're rolling a single dice once
  - we have $\Omega = \{1, 2, 3, 4, 5, 6\} = E$
  - X means evaluating a dice roll
  - ω is the element from our sample space (e.g., we roll a 3)
- we roll twice and want to report the maximum of these two rolls
  - $\Omega = \{(1, 1), (1, 2), (1, 3), \dots, (6, 4), (6, 5), (6, 6)\}$
  - $E = \{1, 2, 3, 4, 5, 6\}$
  - ω is the tuple of the two rolls (e.g., $(3, 5)$)

- x is the value that we assign to this event (in this case, also a 3)
- $x = X(\omega) = 5$ is the realization

Random variables

We can now assign probability functions p to random variables. $p(X = x)$ denotes the probability of the random variable X having the realization x.

The max-of-two-rolls example

- Ω has 36 elements
- $p(X = 1) = \frac{1}{36}$
- $p(X = 2) = \frac{3}{36}$
- ...
- $p(X = 6) = \frac{11}{36}$

In summary, we obtain a **probability distribution** over X, i.e., $p(X)$.

Simplified notation

- use $p(x)$ instead of $p(X = x)$
  - $p(1) = \frac{1}{36}$
  - ...

Note: in the literature, people sometimes use $p(x)$ to also denote the probability distribution over all possible values of x!
$\Rightarrow$ *x is distributed according to p*:

$$x \sim p \quad / \quad x \sim p(x).$$

$\Rightarrow$ *X is distributed according to $p(X)$:*

$$X \sim p(X).$$

Multi-armed bandit notation revisited

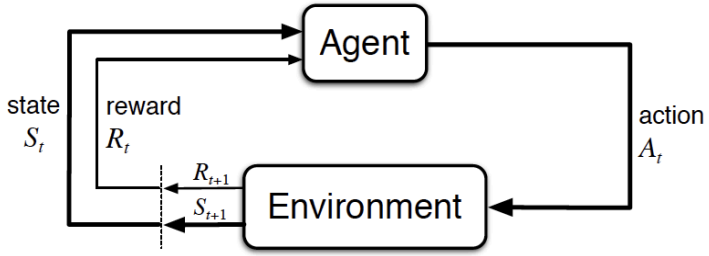
- Let us assume that we have a slot machine and we repeatedly can choose between k different actions.
- After each choice A_t you receive a numerical reward R_t chosen from a stationary probability distribution.
- Objective: maximize the **value**:

$$q(a) = \mathbb{E}\left[R_t | A_t = a\right]$$

.

- We have to rely on estimates $Q_t(a)$ which we can iteratively update based on past experience.

The RL components in extensive notation



Reinforcement Learning framework (Sutton and Barto 1998)

In the literature, we often also find *capital letters* for state, action and reward to properly account for the probabilistic nature

- $p(S_{t+1} = s' | S_t = s, A_t = a)$ is the precise formulation for our short form $p(s'|s, a)$.
- $\pi(A_t = a | S_t = s)$ is the precise formulation for our short form $\pi(a|s)$.

Markov Decision Processes (MDPs)

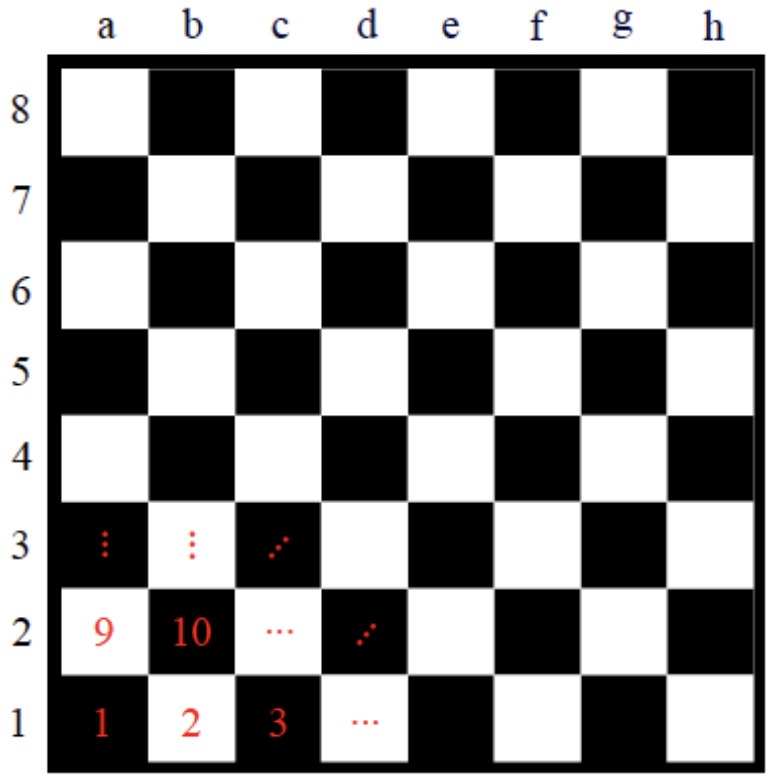
What are MDPs?

- Markov decision processes (MDPs) are a *mathematically idealized form of RL problems*.
- They allow precise theoretical statements (e.g., on optimal solutions).
- Many real-world problems can be abstracted as MDPs.
- In the following we'll focus on:
  - *fully observable MDPs* (i.e., $s_t = o_t$) and
  - *finite MDPs* (i.e., finite number of states & actions).

	All states observable	**Not all states are observable**
No actions	Markov chain	Hidden Markov model
With actions	Markov decision process (MDP)	Partially observable MDP

Scalar and vectorial representation in finite MDPs

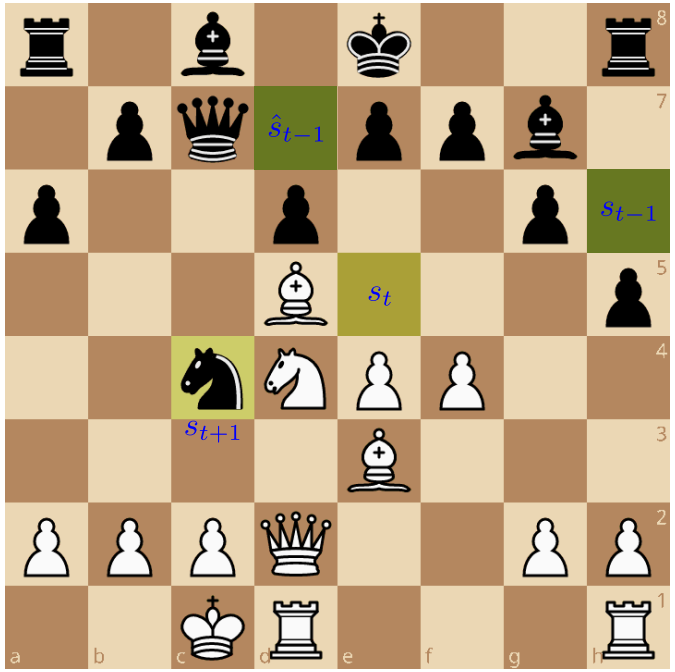
- The position of a chess piece can be represented in two ways:
  - Vectorial: $s = [s_h, s_v]^\top$, i.e., a two-element vector with horizontal and vertical information,
  - Scalar: simple enumeration of all available positions (e.g., $s = 3$).
- Both ways represent the same amount of information.
- **We will stick to the scalar representation of states and actions in finite MDPs.**



Finite Markov chains

A **finite Markov chain** is a tuple $(\mathcal{S}, p)$, where

- $\mathcal{S}$ is a finite set of discrete-time **states** $s_t \in \mathcal{S}$,
- $p(s'|s)$ is the **state transition probability***.



This is a specific stochastic process.

- It creates a **sequence of random variables** $s_t, s_{t+1}, \ldots$.
- It is **memoryless**:
  - the next state only depends on the current state,
  - but it is *independent of past states*:

$$p(s_{t+1}|s_t, s_{t-1}, \ldots, s_0) = p(s_{t+1}|s_t).$$

* $p(s'|s)$ meaning $p(S_{t+1} = s'|S_t = s)$ in extended notation.

State transition matrix

Given a Markov state s and its successor s', the **state transition probability** $\forall\{s, s'\} \in \mathcal{S} \times \mathcal{S}$ is defined by the matrix

$$P = \begin{bmatrix} p_{1,1} & p_{1,2} & \cdots & p_{1,n} \\ p_{2,1} & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & p_{n-1,n} \\ p_{n,1} & \cdots & p_{n,n-1} & p_{n,n} \end{bmatrix}.$$

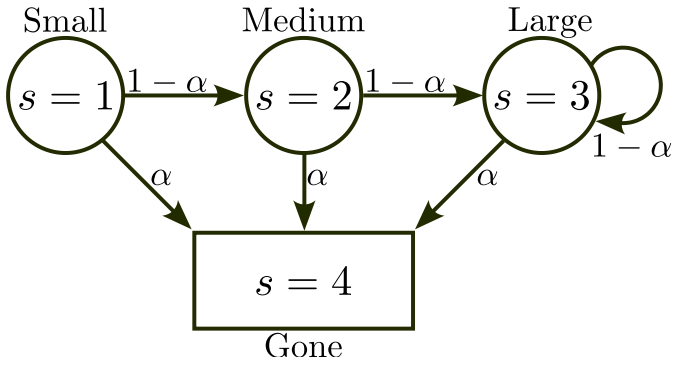
- Here, $p_{i,j} \in [0, 1]$ is the probability to go from state $s = s_i$ to state $s' = s_j$.

- Which property does this matrix need to fulfill?

$$\sum_{j=1}^{n} p_{i,j} = 1 \quad \forall i \in \{1, \dots, n\} \qquad \rightarrow \qquad \text{row stochastic matrix: we need to go } \textit{somewhere}.$$

Example of a Markov chain (1)

- At $s = 1$ a small tree is planted ("starting point").
- A tree grows with probability $1 - \alpha$.
- With probability α, a natural hazard destroys the tree.
- If it reaches $s = 3$ (Large), its growth is limited.
- The state $s = 4$ is terminal ("infinite loop").

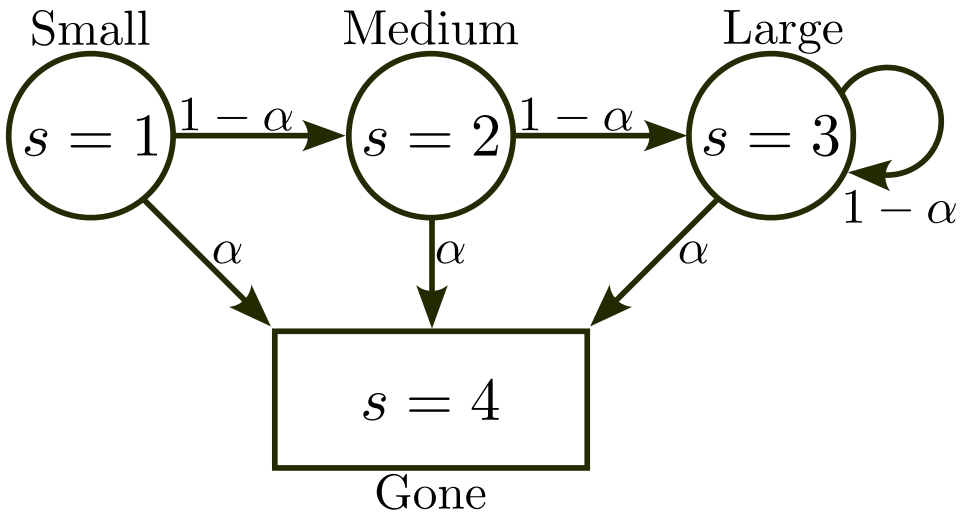


$s \in \{1, 2, 3, 4\} = \{\text{Small, Medium, Large, Gone}\}$

$$P = \begin{bmatrix} 0 & 1-\alpha & 0 & \alpha \\ 0 & 0 & 1-\alpha & \alpha \\ 0 & 0 & 1-\alpha & \alpha \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Source: Oliver Wallscheid's RL course

Example of a Markov chain (2)



Possible **samples** for the given Markov chain example starting at $s = 1$ (small tree):

- Small $\rightarrow$ Gone
- Small $\rightarrow$ Medium $\rightarrow$ Gone
- Small $\rightarrow$ Medium $\rightarrow$ Large $\rightarrow$ Gone

Finite Markov reward processes

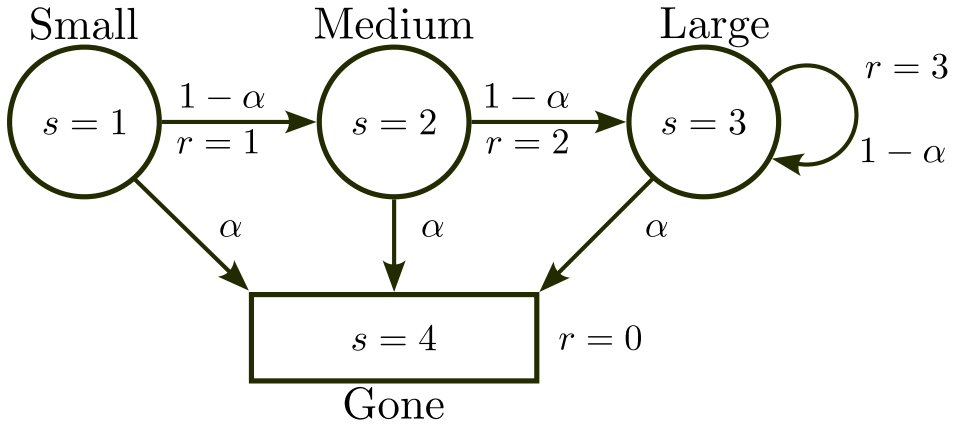
Finite Markov reward processes

A **finite Markov reward process** (**MRP**) is a tuple $(\mathcal{S}, p, r, \gamma)$, where

- $\mathcal{S}$ is a finite set of discrete-time **states** $s_t \in \mathcal{S}$,
- $p(s'|s)$ is the state **transition probability**,
- r is a **reward function** (a random variable with realization $r_{t+1} \sim p(r|s_t)$),
- $\gamma \in [0, 1]$ is a **discount factor**.

- Markov chain extended by rewards.
- Still an autonomous stochastic process, no inputs.
- The reward r_{t+1} only depends on the state s_t.

Example of a finite Markov reward processes



- Growing larger trees is rewarded:
  - advantageous for the environment,
  - appreciated by hikers,
  - useful for wood production.
- Loosing a tree is unrewarded.

In this example, the reward is deterministic, but in general, it is a random variable

Return

The **return** g_t is the *discounted sum of future rewards*, starting from step t.

- For *episodic tasks* ($T < \infty$), this is the finite series

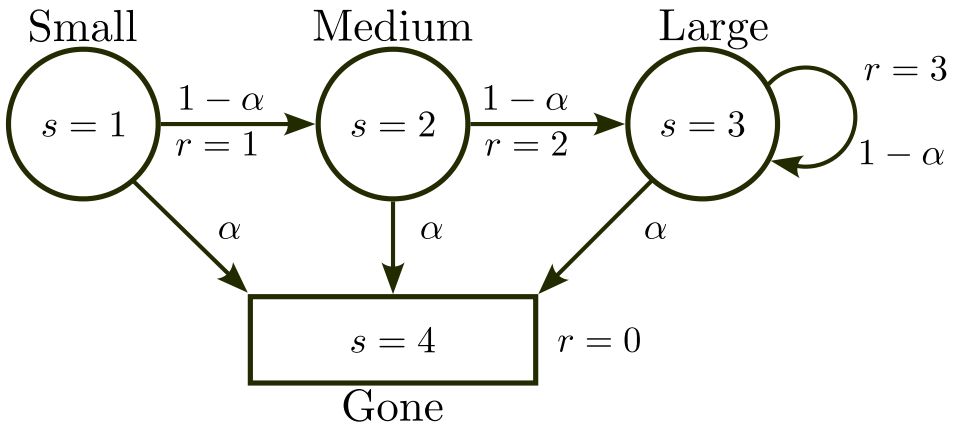
$$g_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots + \gamma^T r_{T+1} = \sum_{k=0}^{T} \gamma^k r_{t+k+1}.$$

- For *continuing tasks*, we get

$$g_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

- The discount factor γ represents the degree to which we value future rewards.
- For $\gamma = 0$, only the immediate reward matters ($0^0 = 1$).
- For $T = \infty$, we need $\gamma < 1$. Why?

Returns for the forest MRP



Exemplary samples for g with $\gamma = 0.5$, starting with the planting ($s = 1$):

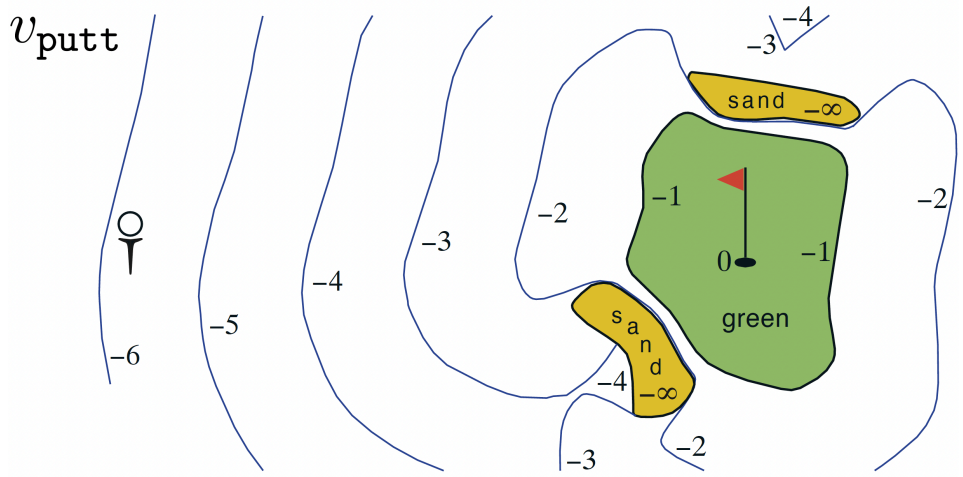
$$
\begin{aligned}
s &= 1 \to 4, & g &= 0, \\
s &= 1 \to 2 \to 4, & g &= 0 + 0.5 \cdot 1 = 0.5, \\
s &= 1 \to 2 \to 3 \to 4, & g &= 0 + 0.5 \cdot 1 + 0.25 \cdot 2 = 1, \\
s &= 1 \to 2 \to 3 \to 3 \to 4, & g &= 0 + 0.5 \cdot 1 + 0.25 \cdot 2 + 0.125 \cdot 3 = 1.375.
\end{aligned}
$$

The Value function

The **state-value function** $V(s)$ (or simply **value function**) of an MRP is the expected return starting from state s at time t:

$$V(s) = \mathbb{E}\left[g_t | s_t = s\right].$$

- Represents the expected long-term value of being in state s.



Exemplary value function for different golf ball locations
(Sutton and Barto (1998))

The Bellmann equation for MRPs (1)

Challenge: How to calculate all state values in closed form?
Solution: The **Bellmann equation**.

$$
\begin{aligned}
V(s_t) &= \mathbb{E}\left[g_t | s_t\right] \\
&= \mathbb{E}\left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots | s_t\right] \\
&= \mathbb{E}\left[r_{t+1} + \gamma\left(r_{t+2} + \gamma r_{t+3} + \ldots\right) | s_t\right] \\
&= \mathbb{E}\left[r_{t+1} + \gamma g_{t+1} | s_t\right] \\
&= \mathbb{E}\left[r_{t+1} + \gamma V(s_{t+1}) | s_t\right] \\
&= \mathbb{E}\left[r_{t+1} | s_t\right] + \gamma \mathbb{E}\left[V(s_{t+1}) | s_t\right]
\end{aligned}
$$

The Bellmann equation for MRPs (2)

Assuming a known expected reward $\hat{r}$ for every state $s \in \mathcal{S}$,

$$\hat{r}_{\mathcal{S}} = \begin{bmatrix} \mathbb{E}\left[r|s_1\right] & \dots & \mathbb{E}\left[r|s_n\right] \end{bmatrix}^\top = \begin{bmatrix} \hat{r}_1 & \dots & \hat{r}_n \end{bmatrix}^\top,$$

for a finite number of n states states with unknown state values

$$V_{\mathcal{S}} = \begin{bmatrix} V(s_1) & \dots & V(s_n) \end{bmatrix}^\top = \begin{bmatrix} V_1 & \dots & V_n \end{bmatrix}^\top,$$

we can derive a linear equation system:

$$V_{\mathcal{S}} = \hat{r}_{\mathcal{S}} + \gamma P V_{\mathcal{S}}$$

$$\begin{bmatrix} V_1 \\ \vdots \\ V_n \end{bmatrix} = \begin{bmatrix} \hat{r}_1 \\ \vdots \\ \hat{r}_n \end{bmatrix} + \gamma \begin{bmatrix} p_{1,1} & \cdots & p_{1,n} \\ \vdots & & \vdots \\ p_{n,1} & \cdots & p_{n,n} \end{bmatrix} \begin{bmatrix} V_1 \\ \vdots \\ V_n \end{bmatrix}$$

Solving the Bellmann equation

$$
\begin{aligned}
& V_{\mathcal{S}} = \hat{r}_{\mathcal{S}} + \gamma P V_{\mathcal{S}} \\
\Leftrightarrow \quad & (I - \gamma P) V_{\mathcal{S}} = \hat{r}_{\mathcal{S}}
\end{aligned}
$$

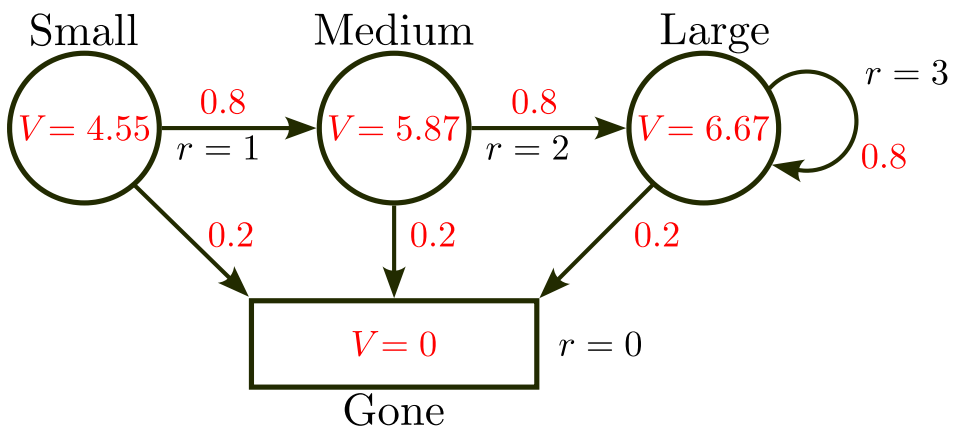
- Possible solutions:
  - direct inversion (Gaussian elimination, $\mathcal{O}(n^3)$),
  - matrix decomposition (QR, Choleskey, etc., $\mathcal{O}(n^3)$),
  - iterative solvers (e.g., Krylov methods, often better than $\mathcal{O}(n^3)$).
- Central challenges in RL:
  - Knowledge of P?
  - Knowledge of $\hat{r}$?
  - Large dimensions!

Example: The value function for the forest MRP

Let's set the disaster probability to $\alpha = 0.2$ and the discount factor to $\gamma = 0.8$.

$$P = \begin{bmatrix} 0 & 0.8 & 0 & 0.2 \\ 0 & 0 & 0.8 & 0.2 \\ 0 & 0 & 0.8 & 0.2 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\begin{aligned} \hat{r}_1 &= 0.2 \cdot 0 + 0.8 \cdot 1 &&= 0.8 \\ \hat{r}_2 &= 0.2 \cdot 0 + 0.8 \cdot 2 &&= 1.6 \\ \hat{r}_3 &= 0.2 \cdot 0 + 0.8 \cdot 3 &&= 2.4 \\ \hat{r}_4 &= 1 \cdot 0 &&= 0 \end{aligned}$$



Markov decision processes

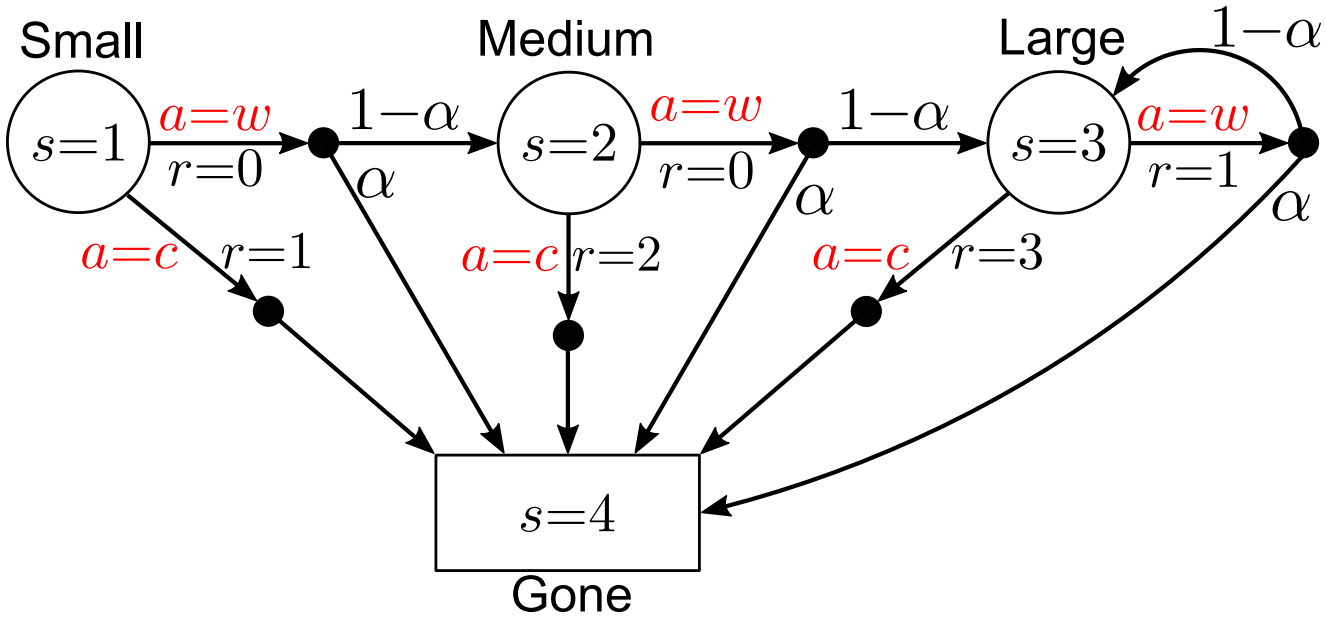
Finite Markov decision processes

A **finite Markov decision process** (**MDP**) is a tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$, where

- $\mathcal{S}$ is a finite set of discrete-time **states** $s_t \in \mathcal{S}$,
- $\mathcal{A}$ is a finite set of discrete-time **actions** $a_t \in \mathcal{A}$,
- $p(s'|s, a)$ is the **state transition probability**,
- r is a **reward function** (a random variable with realization $r_{t+1} \sim p(r|s_t, a_t)$),
- $\gamma \in [0, 1]$ is a **discount factor**.

- Markov reward process extended by actions / decisions.
- Now, rewards also depend on action a_t.

Example of an MDP: Wood cutting



- Two actions possible in each state:
  - Wait $a = w$: let the tree grow.
  - Cut $a = c$: gather the wood.

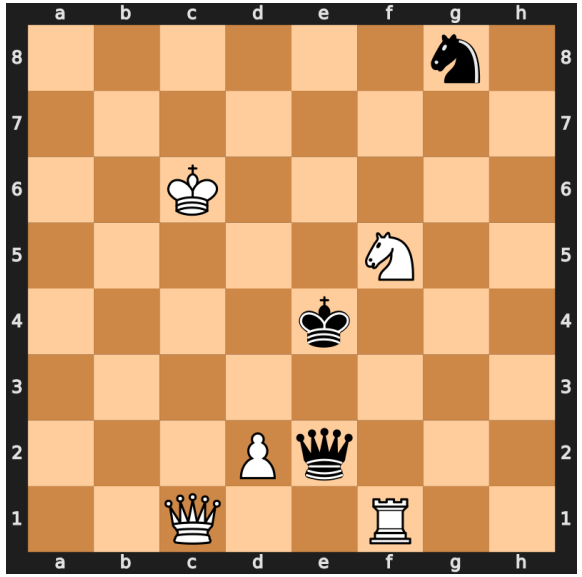
- With increasing tree size the wood reward increases.

Policy (1)

In an MDP environment, a **policy** is a distribution over actions given states:

$$\pi(a_t|s_t) = p(a_t|s_t).^*$$

- In MDPs, policies depend only on the current state.
- A policy completely defines the agent's behavior (which might be stochastic or deterministic).



What's your policy in chess? [Source]

* In extended notation: $p(A_t = a_t|S_t = s_t)$.

Policy (2)

Given a finite MDP $\langle \mathcal{S}, \mathcal{A}, p, r, \gamma \rangle$ and a policy π:

- The state sequence $s_t, s_{t+1}, \ldots$ is a *Markov chain* $(\mathcal{S}, p^\pi)$ since the state transition probability only depends on the state:

$$p^\pi(s'|s) = \sum_{a \in \mathcal{A}} \pi(a|s) p(s'|s, a)$$

- Consequently, the sequence $(s_t, r_t), (s_{t+1}, r_{t+1}), \ldots$ of states and rewards is a *Markov reward process* $(\mathcal{S}, p^\pi, r^\pi, \gamma)$:

$$r^\pi = \sum_{a \in \mathcal{A}} \pi(a|s) p(r|s, a)$$

State-value functions and action-value functions of MDPs

The **state-value function** of an MDP is the expected return starting in s following policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[g_t | s_t = s\right] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t = s\right].$$

The **action-value function** of an MDP is the expected return starting in s, taking action a and then following policy π:

$$Q^\pi(s,a) = \mathbb{E}_\pi\left[g_t | s_t = s, a_t = a\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \middle| s_t = s, a_t = a\right].$$

Bellmann expectation equation (1)

Analog to MRPs, the state value of an MDP can be decomposed into a Bellman notation:

$$V^{\pi}(s_t) = \mathbb{E}_{\pi}\left[r_{t+1} + \gamma V^{\pi}(s_{t+1}) | s_t\right].$$

In finite MDPs, the state value can be directly linked to the action value:

$$V^{\pi}(s_t) \sum_{a \in \mathcal{A}} \pi(a_t | s_t) Q^{\pi}(s_t, a_t).$$

Bellmann expectation equation (2)

Likewise, the action value of an MDP can be decomposed into a Bellman notation:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{\pi}\left[r_{t+1} + \gamma Q^{\pi}(s_{t+1}, a_{t+1}) | s_t, a_t\right].$$

In finite MDPs, the action value can be directly linked to the state value:

$$Q^{\pi}(s_t, a_t) = \mathbb{E}\left[p(r|s_t, a_t)\right] + \gamma \sum_{s_t \in \mathcal{S}} p(s_{t+1}|s_t) V^{\pi}(s_{t+1}). \tag{1}$$

Bellman expectation equation & forest tree example (1)

Let's assume following very simple policy ('fifty-fifty'):

$$\pi(a = \text{cut}|s) = 0.5, \qquad \pi(a = \text{wait}|s) = 0.5 \qquad \forall s \in \mathcal{S}.$$

Applied to the given environment behavior

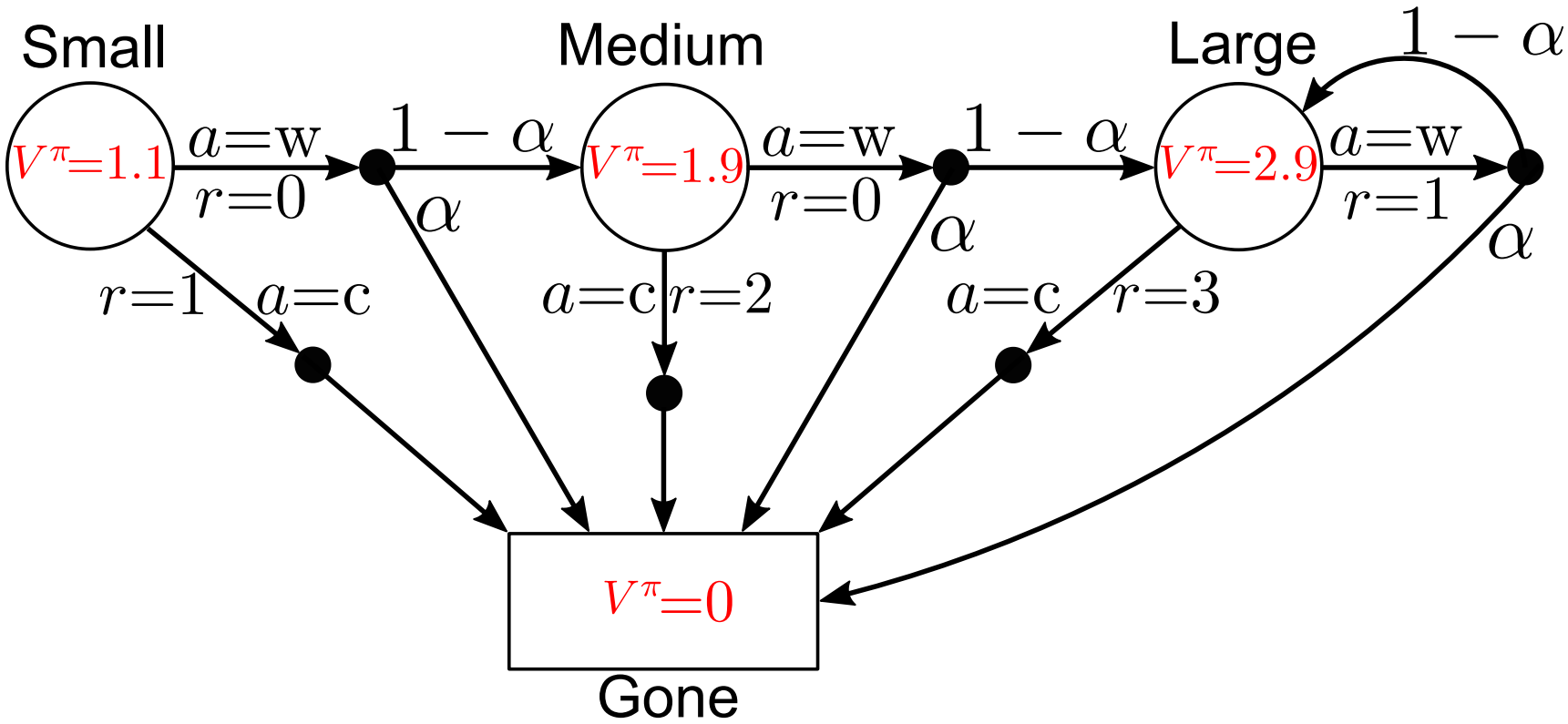
$$p(s'|s, a = \text{cut}) = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad p(s'|s, a = \text{wait}) = \begin{bmatrix} 0 & 1-\alpha & 0 & \alpha \\ 0 & 0 & 1-\alpha & \alpha \\ 0 & 0 & 1-\alpha & \alpha \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\hat{r}^{a=\text{cut}} = \begin{bmatrix} 1 & 2 & 3 & 0 \end{bmatrix}^\top, \qquad \hat{r}^{a=\text{wait}} = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}^\top,$$

We get

$$p^\pi(s'|s) = \begin{bmatrix} 0 & \frac{1-\alpha}{2} & 0 & \frac{1+\alpha}{2} \\ 0 & 0 & \frac{1-\alpha}{2} & \frac{1+\alpha}{2} \\ 0 & 0 & \frac{1-\alpha}{2} & \frac{1+\alpha}{2} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

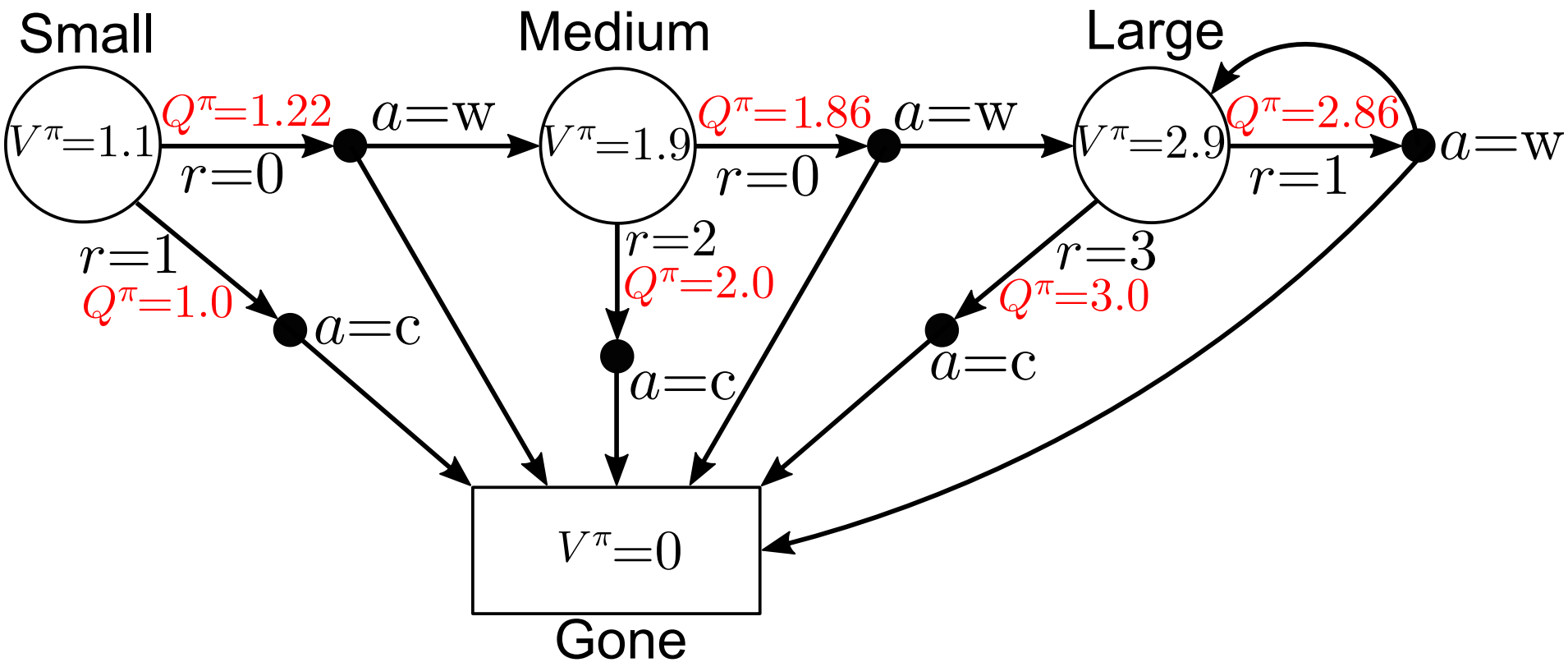
Bellman expectation equation & forest tree example (2)



- Discount factor: $\gamma = 0.8$
- Disaster probability: $\alpha = 0.2$

Bellman expectation equation & forest tree example (3)

Using the Bellmann expectation from (1), the action values can be calculated directly:



Optimal policies and value functions

References

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.